



Multi-Agent Systems

AICB-P2T3 · Ngày 20 · Chương 4 — Agent Nâng Cao

VinUniversity

VinUniversity · Phase 2 · Track 3 · Tuần 4

HÃY SUY NGHĨ...

“Khi một agent không đủ — Supervisor, Debate, Parallel patterns giải quyết bài toán như thế nào?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Tại sao cần nhiều Agent?
2. 5 Agentic Workflow Patterns (Anthropic)
3. Supervisor Pattern — Orchestration
4. Debate Agents — Adversarial Collaboration
5. Parallel Execution & Shared State
6. Multi-Agent Frameworks
7. Demo & Thực hành

Tại sao cần nhiều Agent?

Taxonomy, failure modes, và nguyên tắc “Start simplest”

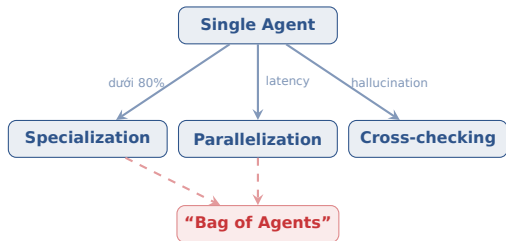
Multi-Agent = Team dự án

Multi-agent giống team dự án: có **PM** (supervisor), **specialists** (workers), **shared document** (state).

Single agent = 1 người làm hết: research, phân tích, viết báo cáo. Tốt cho task đơn giản.

Multi-agent = team 3 người: researcher tìm data, analyst phân tích, writer viết — mỗi người master 1 việc.

Nhưng: Team cũng có overhead — họp, miscommunication, conflict. **Chỉ scale khi 1 người không đủ.**



Nhiều agents + không clear decomposition = chaos

3 lý do chính để multi-agent:

1. **Specialization:** mỗi agent master 1 domain
2. **Parallelization:** concurrent subtasks, giảm latency
3. **Cross-checking:** consensus giảm hallucination

Lưu ý: Decision rule: nếu single agent đạt trên 80% accuracy thì **KHÔNG thêm agents** — complexity không justify.

Use case	Cấu trúc agent	Điểm học viên cần nhớ
Customer support triage	Triage agent route sang Billing / Refund / FAQ agent. Reference: OpenAI Agents SDK handoffs, https://developers.openai.com/api/docs/guides/agents/orchestration	Đây là routing pattern dễ hiểu nhất: mỗi intent có specialist riêng; sai route thì user nhận câu trả lời sai.
Code review assistant	Planner chia task, Code agent sửa, Test agent chạy test, Reviewer agent critique. Reference: GitHub Agent HQ / coding agents discussion, https://github.blog/	Multi-agent hữu ích khi cần plan-implement-verify, nhưng phải có test/CI làm ground truth.
Research report	Searcher thu thập nguồn, Analyst trích insight, Writer viết, Critic fact-check. Reference: Anthropic workflows, https://www.anthropic.com/engineering/building-effective-agents	Phù hợp lab hôm nay vì dễ so sánh single vs multi-agent theo quality, latency, cost.
Enterprise workflow	ADK agent teams phối hợp với tools và observability. Reference: Google ADK, https://adk.dev/	Production không chỉ là prompt: cần deploy, evaluate, trace, auth, guardrails.
Role-based automation	CrewAI crews: role, goal, tools, task. Reference: https://crewai.com/	Tốt để prototype nhanh khi muốn học viên thấy “agent như một vai trò trong team”.
Conversational col	AutoGen group chat / agent	Dùng để minh họa debate

Nhiệm vụ nhóm (8 phút):

1. Với mỗi task, chọn: **single agent**, **workflow**, hay **multi-agent**.
2. Giải thích bottleneck chính: accuracy, latency, cost, hay ownership.
3. Nêu ít nhất 1 lỗi mới nếu dùng nhiều agent.

Task: FAQ đơn giản; research report; refund workflow; code migration.

“Nếu dùng nhiều agent, lỗi mới nào xuất hiện? Nếu dùng một agent, lỗi nào khó kiểm soát?”

Lưu ý: Mục tiêu không phải chọn multi-agent càng nhiều càng tốt; mục tiêu là biết **khi nào không nên dùng**.

14

Failure modes
identified

3

Nhóm lỗi:
spec, align-
ment, verification

150+

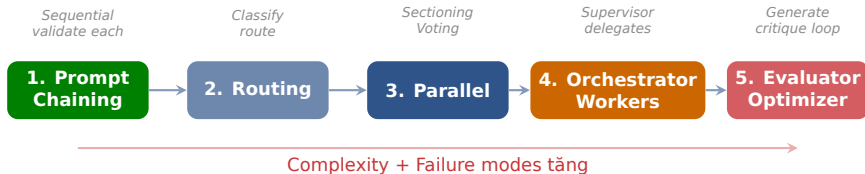
Tasks được
phân tích

“Why Do Multi-Agent LLM Systems Fail?” (arXiv:2503.13657) chỉ ra nhiều lỗi đến từ **specification / system design, inter-agent misalignment, và verification / termination**. Vì vậy cần define roles, stop condition, và rubric trước khi code.

5 Agentic Workflow Patterns (Anthropic)

Từ đơn giản đến phức tạp — escalate dần

02



“Start simplest. Only add agents when **measurably** needed.” Thử Prompt Chaining trước. Chỉ escalate khi có bằng chứng cần.

Sectioning: split tasks → parallel workers

Voting: same task → multiple LLMs → aggregate

Chia nhóm 3 người, mỗi nhóm nhận 1 case:

- Tổng hợp review khách hàng từ 1.000 comment
- Xử lý ticket refund / billing / technical support
- Viết báo cáo thị trường có fact-check nguồn
- Migrate codebase và đảm bảo test pass

Nhiệm vụ: chọn 1 trong 5 patterns, vẽ flow trong 4 phút.

Mỗi nhóm nói 30 giây: pattern, lý do, metric đo thành công, failure guard.

Lưu ý: Không được mặc định “supervisor cho mọi thứ”. Hãy chứng minh pattern đơn giản hơn chưa đủ.

Routing — Classify input rồi chuyển đến specialized handler. Easy queries dùng small model, hard queries dùng large model.

- 70% queries easy = small model (GPT-4o-mini)
- 30% queries hard = large model (GPT-4o)
- Giảm 50%+ chi phí tổng thể

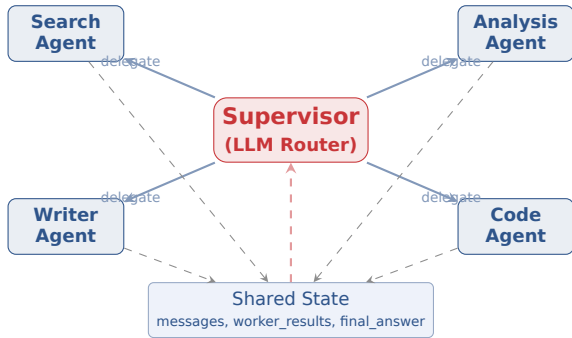
Khi workload có **bimodal difficulty** — nhiều query đơn giản + ít query phức tạp.

Lưu ý: Routing chỉ hiệu quả khi phân loại chính xác. Sai phân loại = query hard gửi cho small model = kết quả kém.

Supervisor Pattern — Orchestration

Hub-spoke delegation với LangGraph

03



Supervisor — LLM router: nhận task, decompose, route đến workers, aggregate results

- Mỗi worker là node riêng với own tools
- Supervisor quyết định: gọi ai, theo thứ tự nào
- **Cost insight:** cheap model cho routing, expensive model cho workers

```
class SupervisorState(TypedDict):
    messages: list[BaseMessage]
    next_worker: str
    worker_results: dict[str, str]
    final_answer: str

# Supervisor routing via tool calls
def supervisor(state):
    response = llm.invoke(
        system="Route task to workers",
        tools=[search, analyze, write],
        messages=state["messages"]
    )
    return {"next_worker": response.tool}

# Build graph
graph = StateGraph(SupervisorState)
graph.add_node("supervisor", supervisor)
graph.add_node("search", search_agent)
graph.add_node("analyze", analysis_agent)
```

State gồm 4 thành phần:

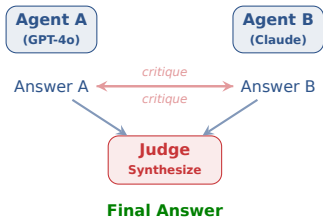
1. messages: conversation context
2. next_worker: ai được gọi tiếp
3. worker_results: output từ mỗi worker
4. final_answer: kết quả tổng hợp

Lưu ý: Failure modes: **#1** infinite routing loop ($A \rightarrow B \rightarrow A$). **#2** wrong worker selection. Cần **max iterations** guard.

Debate Agents — Adversarial Collaboration

Giảm hallucination qua tranh luận có kiểm soát





Flow:

1. Agent A & B answer **independently**
2. Critique nhau (adversarial)
3. Judge synthesize final answer

Debate giảm hallucination **15-25%**. Hiệu quả nhất cho ambiguous queries, high-stakes decisions.

Lưu ý: “Collective delusion”: cả hai đồng ý sai → Judge không catch. Fix: dùng **diverse models**.

Agent	Model	Strength
Researcher	GPT-4o	Broad knowledge
Analyst	Claude	Nuanced reasoning
Critic	Gemini	Different training data
Judge	Best-of	Final synthesis

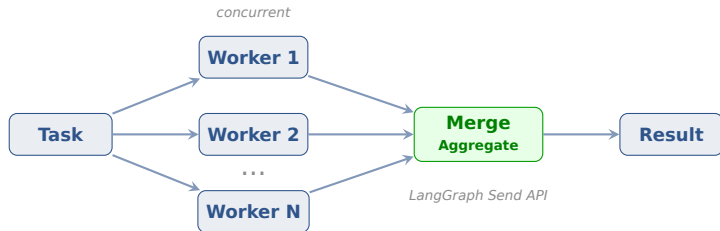
Mỗi model có **blind spots khác nhau** do training data diversity.

GPT-4 + Claude + Gemini
→ better coverage hơn 3 instances GPT-4.

Trade-off: thêm latency (sequential critique) + cost (3 models). Chỉ justify cho high-stakes.

Parallel Execution & Shared State

Map-reduce, AsyncIO, và coordination patterns



Dynamic fan-out + parallel branches — built-in parallelism. Mỗi worker chạy concurrent, merge khi tất cả done.

Lưu ý: “Parallel $\neq$ always faster”: DAG dependencies, shared state locks, merge conflicts có thể negate speedup. Đo trước khi assume.

Shared state (blackboard) —

Agents read/write central state — simple, cần locks cho concurrent writes. LangGraph dùng TypedDict state mặc định.

Message passing — Async queue (Redis Pub/Sub, Kafka) — decoupled, scalable, thêm network latency. Tốt cho distributed systems.

Lưu ý: Context loss across hand-offs: biggest coordination failure. Mỗi agent nhận partial context, quality giảm.

Multi-agent without tracing = impossible to debug. Dùng LangSmith hoặc Langfuse cho mọi multi-agent system.

Failure handling: supervisor detect timeout thì retry different worker hoặc fallback.

True concurrent execution cho multiple agents.

```
asyncio.gather(agent1(), agent2())
```

→ Parallel API calls, shared event loop

Pre-initialize N instances, use queue

→ Amortize init cost, control concurrency

Giống thread pool nhưng cho LLM agents

Production failure handling:

- Supervisor detect **timeout** → retry hoặc fallback
- **Circuit breaker**: fail 3 lần → skip, log, backup
- **Dead letter queue**: failed tasks lưu lại để debug

Lưu ý: Pool size = concurrent API rate limit. Quá nhiều agents → rate limit errors.

Multi-Agent Frameworks

LangGraph, AutoGen, CrewAI — chọn đúng công cụ

Framework	Flexibility	Setup	Best for	Ghi chú
LangGraph	Cao	Trung bình	Production	State machines, full control
CrewAI	Trung bình	Dễ	Prototype	Role-based, nhanh onboard
AutoGen	Cao	Trung bình	Code exec	GroupChat, human-in-loop
OpenAI SDK	Thấp	Rất dễ	Simple	Lightweight, tool-use focused
Google ADK	Trung bình	Trung bình	GCP	Vertex AI, A2A protocol

AutoGen/CrewAI tốt cho prototype nhanh. **LangGraph cho production** — full state control, debugging, conditional routing. Migrate khi cần scale.

Demo & Thực hành

Multi-Agent Research System + 2 giờ lab có peer review



Cho học viên xem task: “Research GraphRAG state-of-the-art, write 500-word summary”. Hỏi: agent nào nên chạy trước? bước nào có thể parallel?

So sánh dự đoán với LangSmith trace: route có đúng không? worker nào tốn token nhất? lỗi nào cần guardrail?

Biến demo từ “giảng viên biểu diễn” thành “học viên debug một hệ thống thật”.

1. Pipeline: Supervisor + SearchAgent + AnalysisAgent + WriterAgent
2. Task: “Research GraphRAG state-of-the-art, write 500-word summary” — supervisor coordinates
3. LangSmith trace: routing decisions, parallel timeline, worker outputs, final synthesis
4. So sánh: single-agent vs multi-agent trên cùng task — quality, latency, cost

Mục tiêu: Build 3-agent research system: Researcher + Analyst + Writer với LangGraph

Deliverable: Benchmark report: single-agent vs multi-agent (accuracy, latency, cost) + LangSmith traces

Thời gian: 2 giờ

- 1. 0-15'** Setup: clone starter, set API key, chạy single-agent baseline.
- 2. 15-45'** Build supervisor: LangGraph state machine, routing via tool calls.
- 3. 45-75'** Add 3 workers: SearchAgent, AnalysisAgent, WriterAgent; lưu output vào shared state.
- 4. 75-95'** Trace & benchmark: single vs multi trên 3-5 research queries; đo quality, latency, cost.
- 5. 95-115'** Peer review: đổi nhóm, đọc trace của nhau, tìm 1 failure mode và đề xuất fix.
- 6. 115-120'** Exit ticket: mỗi nhóm ghi 1 điều nên dùng multi-agent và 1 điều không nên dùng.

GitHub repo + benchmark report + 1 LangSmith/Langfuse trace screenshot.
Bonus: thêm Critic Agent để fact-check.

Tiêu chí	Câu hỏi peer reviewer phải trả lời	Điểm
Role clarity	Mỗi agent có nhiệm vụ rõ, không overlap quá nhiều không?	0-2
State design	Shared state có đủ thông tin để handoff mà không mất context không?	0-2
Failure guard	Có max iterations, timeout, retry/fallback, hoặc validation không?	0-2
Benchmark	Có so sánh single vs multi-agent bằng metric cụ thể không?	0-2
Trace explanation	Nhóm giải thích được trace: ai làm gì, tốn bao nhiêu, sai ở đâu không?	0-2

Mỗi nhóm review 1 nhóm khác trong 8 phút, sau đó owner có 5 phút sửa nhanh.

10 câu trắc nghiệm + short answer:

Scope: Reflexion, Memory Systems, Production RAG, GraphRAG, Multi-Agent

Format: 7 MC + 3 short answer

Câu hỏi test understanding, không test nhớ syntax

Submit portfolio từ N16-N20:

- Lab 16: Reflexion agent
- Lab 17: Memory system
- Lab 18: Production RAG
- Lab 19: GraphRAG
- Lab 20: Multi-agent system

Deadline: 1 tuần sau N20

Anthropic: Building effective agents
[anthropic.com/engineering/building-effective](https://anthropic.com/engineering/building-effective-agents)

OpenAI: Agents SDK orchestration
+ handoffs
developers.openai.com/.../orchestration

AutoGen:
microsoft.github.io/autogen

CrewAI: crewai.com

Google ADK: adk.dev

LangGraph supervisor library
reference.langchain.com/python/langgraph-supervisor/

MAST paper: Why Do Multi-Agent LLM
Systems Fail?
arxiv.org/abs/2503.13657

Lưu ý: Gợi ý đọc: học viên chỉ cần đọc Anthropic + OpenAI handoffs trước lab; các link còn lại dùng khi chọn framework cho project.

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 Supervisor là most practical multi-agent architecture — clear ownership, easy debug, production-ready
- 2 Debate giảm hallucination 15-25% nhưng tốn $2-3\times$ cost — chỉ dùng cho high-stakes decisions
- 3 Parallel execution cắt latency nhưng cần careful state merge strategy — “parallel $\neq$ always faster”
- 4 Buổi học giữ lý thuyết trong 2 giờ; 2 giờ còn lại build, trace, benchmark, peer review

Ngày 21: Fine-tuning LLMs — LoRA/QLoRA

“Khi nào nên fine-tune — và khi nào prompt engineering đủ rồi?”

- Hoàn thành Lab 20 + submit Milestone 1 portfolio
- Đọc: MAST paper (NeurIPS 2025) — multi-agent failure taxonomy

Hỏi & Đáp

Supervisor vs Debate vs Parallel — khi nào dùng pattern nào? Single agent có khi nào đủ?



Cảm ơn!

AICB-P2T3 · Ngày 20 · Multi-Agent Systems

github.com/vinuni-aicb

Liên hệ: instructor@vinuni.edu.vn